

DESE5001 - Data Science

Assignment 2: Final Group Report 2024

Using existing weather conditions to predict rain probability the next day at Sydney Airport



Figure 1: Heavy rainfall in Sydney [1]



1. Abstract

Our group applied four different machine learning algorithms to the dataset: Support Vector Machine, Logistic Regression, Decision Trees and Random Forest. The purpose of these models would be to predict if it would rain on the following day, given the weather data of the current day.

Logistic Regression appeared to have the best accuracy and recall. However, since each model focuses on different subsets of the data, their validity cannot be extended to predict diagnoses outside their specific scope.

2. Introduction

Rain prediction plays a vital role in various sectors by enhancing safety, efficiency, and preparedness across different aspects of life and industry. It is especially crucial for areas surrounding airports to maintain efficient, safe, and reliable airport operations. Rain predictions are predictable through analysis tools and advanced technologies, such as weather satellites and radar systems, but they are often expensive and have resolution limits. Data analysis of weather features could provide a new method for early forecast of rain probability.

Machine learning is often used in weather forecasts to analyse vast amounts of weather data to improve prediction accuracy and identify subtle patterns that traditional methods might miss.

3. Acquiring and Preprocessing Data

In the section below we state our dataset source, size and number of attributes as well

as the methods used to better prepare the data. (Filtering, Balancing & Splitting the data)

3.1 Chosen Data

We selected our dataset from kaggle [2]. The dataset is made up of '10 years of daily weather observations from numerous locations across Australia' [2]. It contains 144,461 samples and 20 attributes. It should be noted the data set is observed data from 11.01.2007 to 25.06.2017.

3.2 Extracting and cleaning the required data from dataset

First, we examined the dataset to identify any missing values. We found various attributes where values had been recorded as N/A on certain days.

To ensure we had usable values for each attribute on any given day, we filtered out entries marked as N/A. This refinement was feasible due to the high volume of data within the sample, allowing us to not be limited in the set of attributes available to us.

Next, we evaluated the remaining data. The dataset was organised by location first and then date. After assessing which location retained the most data, we selected Sydney Airport, with 1433 data samples, as the location of choice.

Sydney Airport likely had the most dates remaining because such variables are recorded out of necessity to monitor flying conditions.

3.3 Numerising the data

The majority of the dataset is numerical apart from the wind data (prevailing / gusts etc), which were represented in the format of a 16 wind compass rose consisting of the four cardinal directions, four intercardinal directions and 8 half-winds (i.e NNE, ENE etc), and the

two binary “did it rain” features (represented with “Yes” and “No”).

The wind directions were converted into their respective bearings in degrees, and the “did it rain” features were converted to 1 and 0 respectively, in Microsoft Excel (formula in appendix).

3.4 Adjusting Data Skew

It was found that the data resulting from the above processing was skewed towards the negative outcome of the independent variable, with 718 data points for will it rain = “yes” and over 2168 data points for will it rain = “no”.

To fix this, the cleaned data was imported into matlab and an equal number of “no” data points to the number of “yes” data points were selected randomly and this was combined with the “yes” datapoints to obtain the final balanced dataset.

3.5 Standardising dataset

Different features had large variations of means and standard deviations, so all the non-binary features were scaled and adjusted using Matlab’s `normalize()` function.

This final dataset comprises 20 features and 1429 data points of purely numerical data.

3.6 Splitting dataset

The final dataset obtained above was split into training, testing and validation data in the ratio 8:1:1 respectively.

4. Analysis and Results

4.1 Decision trees

4.1.1 Methodology

4.1.1.1 Introduction

Decision trees are a fundamental example of a non-linear model within the realm of supervised learning algorithms. When using decision trees, data is split into subsets from the root node, where each internal node represents a decision based on an attribute and each leaf node represents the result of that decision [3]. When utilising it for classification tasks the most common criteria considered is the gini number.

Advantages of decision trees include the fact that the visual representation of all nodes and branches are easy to interpret, and that all features can be both discrete and continuous.

As we will see later in the report, a disadvantage of this method is its tendency to overfit the training data.

4.1.1.2 Hyper-parameter optimization

The pre-processing of the data was done as a group to ensure the data we were using was the same data for training, testing and validation. The data was split into three csv files, so the only set-up required was assigning the dependent variable (‘whether it will rain tomorrow’) to ‘y’ for each of the training, validation and testing datasets, and ‘x’ to all the other variables.

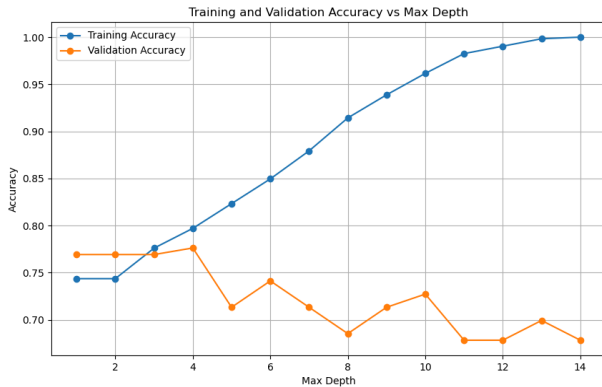


Figure 2: Graph of Max depth vs Accuracy

In order to observe how the accuracy of the model is affected by the number of layers allowed within the tree, a comparative graph was formulated. If this is small, underfitting occurs as the model is too simple to recognise patterns within the data. This is evident in the graph above, where the validation accuracy is higher than that of the training data. We can also see that as the depth increases further, major overfitting occurs, so the optimal max depth value chosen was 3

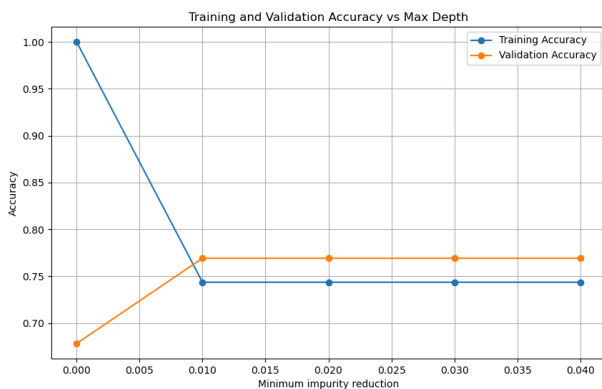


Figure 3: Graph of Min impurity decrease vs Accuracy

For this model the minimum impurity decrease refers to the minimum reduction in the gini number for a node to be split. The gini impurity measures the probability a value would be wrongly classified if labelled at random. A lower impurity decrease leads to more splits and a more complex tree as there is less of a threshold for the node to split. This can often lead to overfitting, as can be seen for very

small impurity decreases in the graph above. The optimal value selected to use in the model was 0.01.

4.1.2 Results

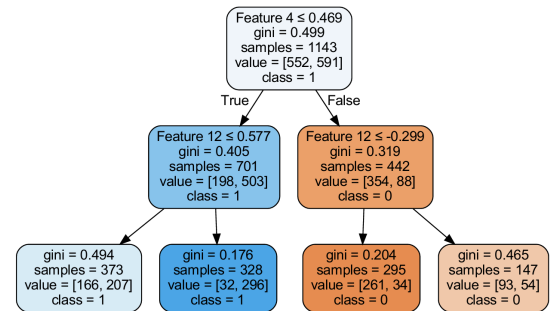


Figure 4: Decision tree

	Accuracy	Precision	Recall
Training	1	1	1
Validation	0.6713	0.6670	0.671
Testing	0.7412	0.7617	0.7413

Table 1: Accuracy, precision & recall for split data

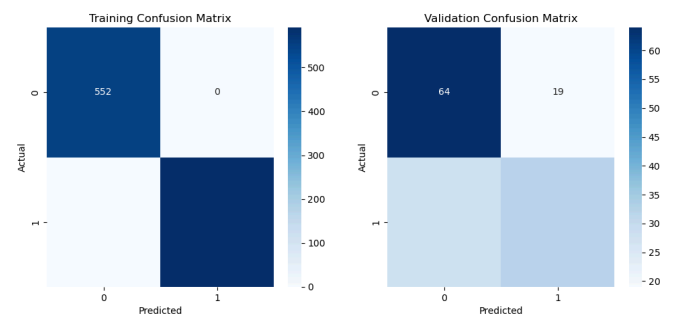


Figure 5: confusion matrix for training and validation data

The confusion matrix shows 522 True negatives for the training data and 64 & 19, True negatives and False positives respectively for the validation data

4.1.3 Discussion

The training data was calculated by the model to have 100% accuracy, precision and recall. This is highly likely to be caused by overfitting, as there is a large drop in accuracy for the validation data. The depth of the tree does not consist of that many layers, so 'pruning' isn't required. Feature selection would effectively ensure the model only retains the most relevant and powerful features. Also increasing the number of data values included in the training data would allow the model to generalise better and learn more relevant patterns. A feature heatmap is considered and visualised in the conclusion. It is also important to note that outliers were not removed from the dataset and these can heavily affect the outcome of the decision tree. They can lead to splits that are not representative of the general data distribution as extreme values can influence the split more than general patterns

4.2 Support Vector Machine

4.2.1 Methodology

An SVM model was trained and optimised on matlab. The purpose of this model is to predict whether it would rain on the following day, given the weather conditions on the current day. Sequential feature selection was also performed to maximise accuracy.

4.2.1.2 Sequential Feature Selection

Forward sequential feature selection was performed by evaluating a model (initially only on one feature) and sequentially adding in a new feature (the one with the next highest correlation to the dependent variable) and calculating the classification loss (1-accuracy of

model) until the classification loss stopped decreasing.

Thus, the features (arranged here in order of feature importance (ie how much it contributes to predicting the dependent variable)) cloudiness at 3pm, pressure at 3pm, humidity at 9am, wind gust speed and sunshine were selected to train the final model.

4.2.1.2 Hyper-parameter optimization

Hyper-parameter optimization was performed (in parallel to speed up computation) on the model using the Bayesian Optimization algorithm. This allows optimal kernel scale and box constraint values to be selected.

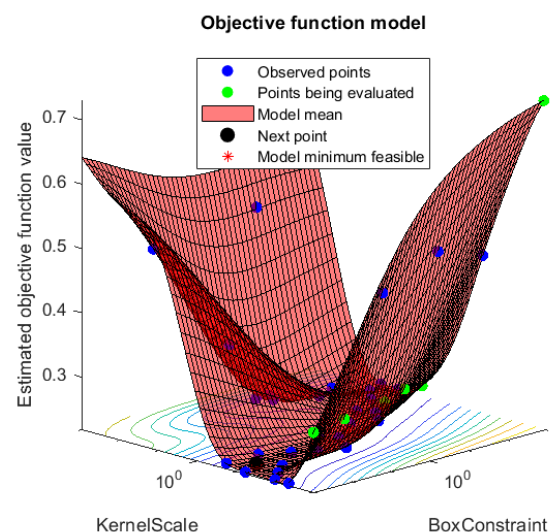


Figure 6: Graph of loss vs kernel scale and BoxConstraint

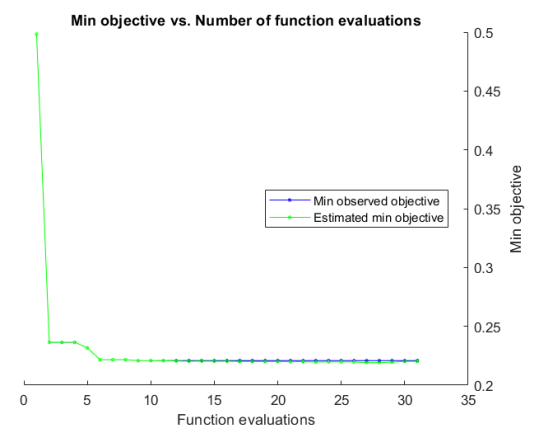


Figure 7: Graph of loss vs iterations of the hyperparameter tuning algorithm.

A linear kernel function was selected, and the results for the best estimated BoxConstraint is 0.08053 and best estimated KernelScale is 0.061235 over 31 iterations.

4.2.2 Results

The hyperparameter optimization showed a minimum classification loss of 0.22084, which is an accuracy of 77.9%.

The model was then fed with unseen labelled training data and the resulting confusion matrix is shown:

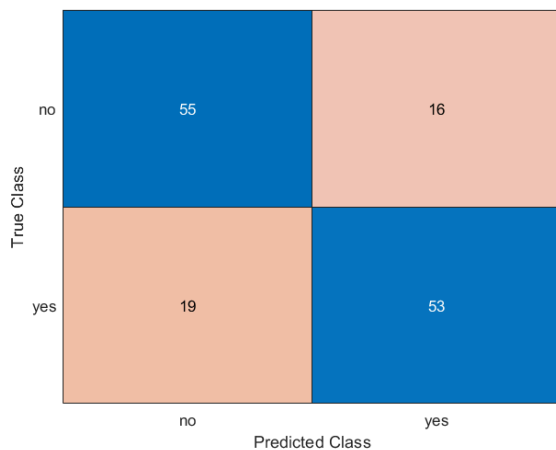


Figure 8: Confusion matrix

Which yields the following information about the model's performance:

Accuracy	75.5%
Precision	76.8%
Recall	73.6%

Table 2: Accuracy, precision & recall for split data

Additionally, the training data was fed back into the model (to check for overfitting) as unseen labelled data and an accuracy of 78.1% was observed.

4.2.3 Discussion

The feature importance (given in section 4.2.1.2) makes intuitive sense, as the evening (3pm) data for cloudiness and pressure would be a good indicator for rain the next day.

The model exhibits a mediocre performance, which could potentially be improved with more data points. The model does exhibit a small degree of overfitting as the accuracy for the training data (78.1%) is slightly above the accuracy of the testing data (75.5%).

4.3 Random Forest

4.3.1 Methodology

The random forest model is an ensemble machine learning algorithm known for its high accuracy and stability.[4] It operates by constructing multiple decision trees during training and outputting the mode of the classes of the individual trees. The model is trained and optimised using python.

The model has been applied to the dataset to predict whether it will rain tomorrow at Sydney Airport. This data can be crucial for planning, operational, activities and maintenance applications at an airport.

4.3.1.1 Method Justification

The model does not assume linearity between variables, allowing it to model complex relationships within the data. The model is good at handling the diverse weather variables at Sydney Airport, providing good predictions even in the presence of outliers. The model's ability to assess the feature's importance is crucial for prioritising the most impactful weather variables.

4.3.1.2 Data pre-processing and cleaning

Firstly, a seaborn heatmap was used to identify correlations that were to be considered for redundancy. High positive and negative correlated variables with each other and not the target were removed. Although, Random forest models are less sensitive to multicollinearity, reducing redundancy can help with interpretability and reducing overfitting.

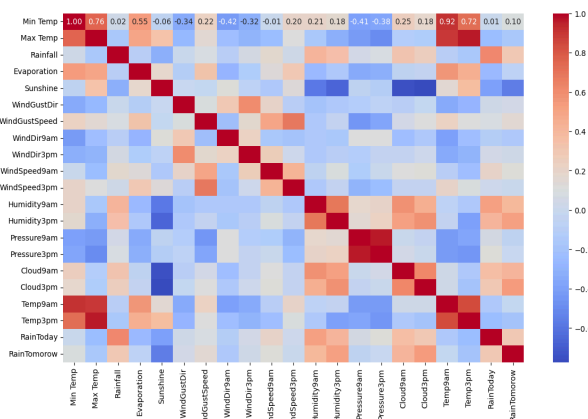


Figure 9: Seaborn Feature correlation heatmap

The heatmap showed there is a high correlation between 'Max Temp' and 'Min Temp' (0.75) and also with the pressure readings 'pressure 9am' and 'pressure 3pm' (0.91). The column 'Min Temp' and 'pressure 9am' was dropped due to the latter being able to capture more variance in temperature changes. 5 more variables were also dropped, making the regression 14 variables.

4.3.1.3 Hyper-parameter optimisation

Four key features from the random forest ('max_depth', 'min_sample_split', 'min_impurity_decrease', 'n_estimators') Plots for each of the key features were made to compare the accuracy of the validation and training data. The models were optimised to prevent overfitting or underfitting within the model. Each optimised hyperparameter was iterated into the next model.

4.3.1.3.1 Maximum tree depth

Maximum depth is related to the roots and the leaves of the tree. In general, more roots and

leaves of the tree can result in better classification, however, the more the max depth increases the model becomes more susceptible to overfitting.

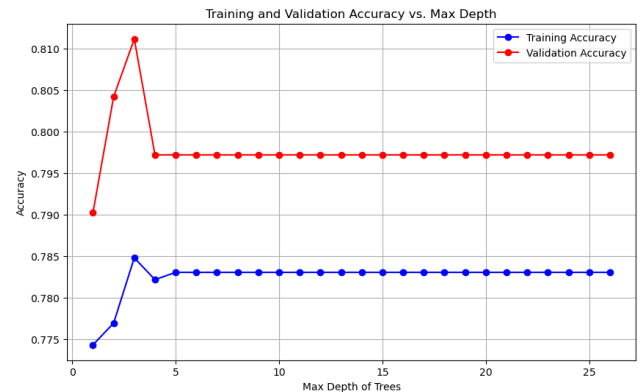


Figure 10: validation and accuracy vs max depth

It can be seen from figure10 that the training accuracy shows a slight increase that stabilises around a depth of 3, the validation accuracy reaches its peak at a depth of 3 and then remains consistent. The max depth was chosen as 3 as it prevents overfitting while maintaining good generalisation to the data.

4.3.1.3.2 Minimum Sample Split

The minimum sample split is the minimum number of samples required to split an internal node.[5]

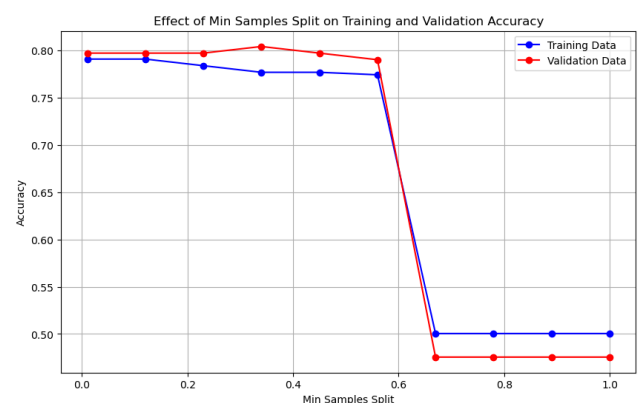


Figure 11: min sampling vs training and validation

Figure11 shows the minimum sample split to be around 0.1 where the validation accuracy is maximised while maintaining a high training accuracy. This range provides a good balance

by preventing the model from being too complex which would lead to overfitting if the value is too low and not too simplistic which would lead to underfitting if it is too high. The selected 'min_sample_split'=0.1.

4.3.1.3.3 Minimum Impurity Decrease

The Minimum impurity decrease is a threshold for making a split at a node in a decision tree, where a split will only occur if it decreases the impurity by at least this value.[6]

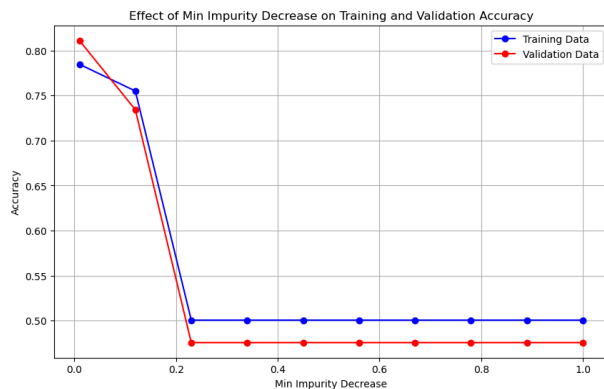


Figure 12: training and validation vs min impurity decrease

The results show that training and validation accuracies decrease rapidly as the min_impurity_decrease increases from 0 to roughly 0.1, suggesting that choosing a higher threshold for impurity reduction results in models that miss important patterns in the data.

The accuracy levels plateau below a min_impurity_decrease of 0.1, indicating that additional increases in this threshold are unlikely to have a substantial impact on the model since it is likely to stop splitting, leaving a very simplistic model that is unable to learn from the data. Thus, a minimum impurity decrease is set to 0.

4.3.1.3.4 Number of Estimators

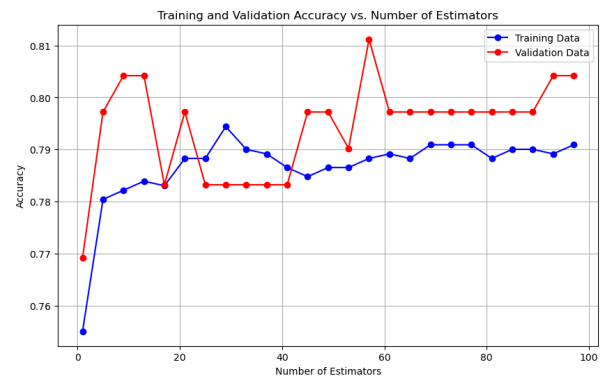


Figure 13: Training and validation vs n_estimators

The n_estimator is the number of trees represented in the forest.

The training accuracy shows minor fluctuations with a general upward trend, suggesting incremental learning gains with additional trees.

In contrast, the validation accuracy stabilises around 80 estimators, indicating optimal generalisation capacity without significant overfitting. Thus, 80 estimators is selected.

4.3.1.3.5 Final model fit values.

The final model had the parameters model = RandomForestClassifier(n_estimators=80,max_depth=3,min_samples_split=0.1,min_impurity_decrease=0,random_state=0).

4.3.2 Results

The tree has 3 estimators alongside its single tree, this effectively introduces randomness and diversity into the model.

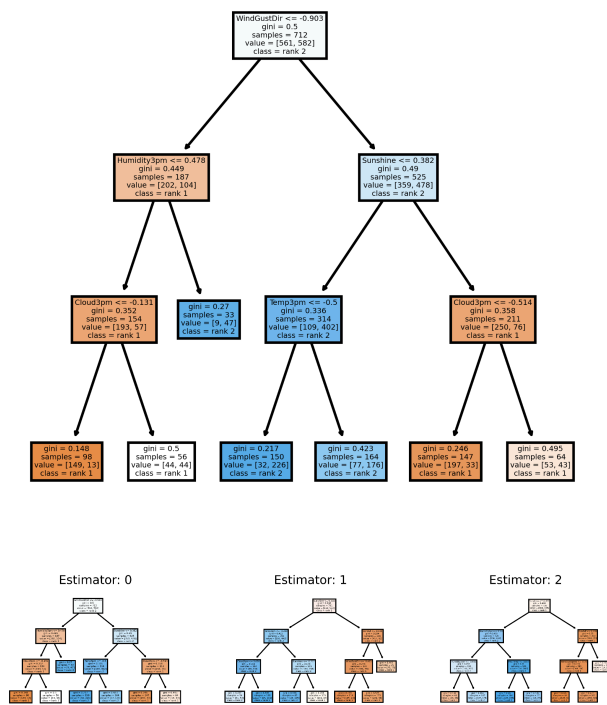


Figure 14: Tree and Random forest also in appendix

Model	Training	Validation	Test
1	0.785	0.790	0.669
2	0.791	0.797	0.669
3	0.785	0.811	0.699
4	0.801	0.804	0.678

Table 3:Initial model results

The results indicate that while models 1, 2, and 3 show relatively consistent accuracy between training and validation, all models experience a notable drop in test accuracy, suggesting they may be overfitting and not generalising well to unseen data. Techniques such as cross-validation can be used for fine-tuning, using regularisation methods to constrain model complexity

4.3.1.3 Final results

	Accuracy	Precision	Recall
Training	0.801	0.801	0.801
Validation	0.804	0.805	0.806
Testing	0.678	0.678	0.675

Table 4: Final model results

The model exhibits strong performance on the training and validation datasets with accuracy, precision, and recall all above 0.8, indicating good fit and balance in handling the positive class. However, there is a significant drop in all three metrics on the test set, with values around 0.678, suggesting the model may not be generalising well to unseen data, potentially due to overfitting. To improve test performance, further exploration of model parameters, better feature selection, or the use of more generalised or regularised models could be considered.

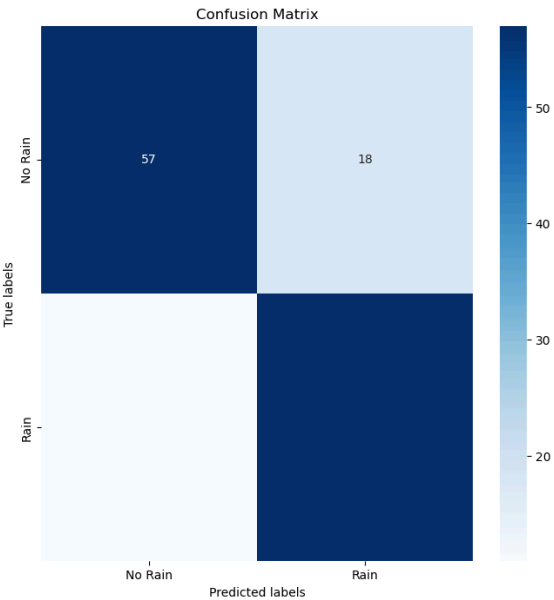


Figure 15:Confusion matrix for will it rain

A confusion matrix has been modelled and can display a clear colour difference between the rain and no rain signifying that the model is predicting the results correctly.

4.3.3 Discussion

A feature importance plot was plotted to identify the most significant contributing variables to the model. The highest contributors were identified as Sunshine indicating that if all these variables have relevant values it is highly likely that it will rain tomorrow.

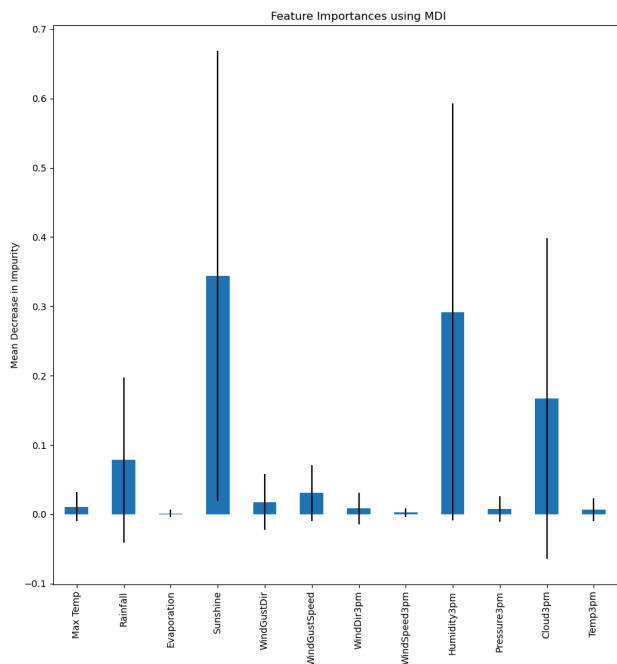


Figure 16: Feature Importance graph

To improve the model a GridSearchCV can be used to tune the hyperparameter. Instead of iterating some values, the values can be done automatically. The GridSearchCV was tested and it gave a Validation Accuracy of Best Parameters: 0.8251748251748252. This method is efficient, by suggesting the most relevant features the method can reduce overfitting.

4.4 Logistic Regression-

Predicting probability of rain the next day if it rained the previous day

4.4.1 Methodology

Logistic regression was selected because it is a statistical model that identifies the relationship between independent variables and a binary dependent variable - such as predicting whether it will rain tomorrow (a binary outcome: rain or no rain).

Initially, a feature correlation map was plotted to visually represent the strength and direction of relationships between different variables. This helped in identifying which variables are strongly correlated with each other and with the target variable (rain tomorrow), providing insights into the underlying patterns and relationships. Additionally, by focusing on the most relevant features (those with strong correlations with the target variable and weak correlations with each other), a more accurate and efficient model could be built.

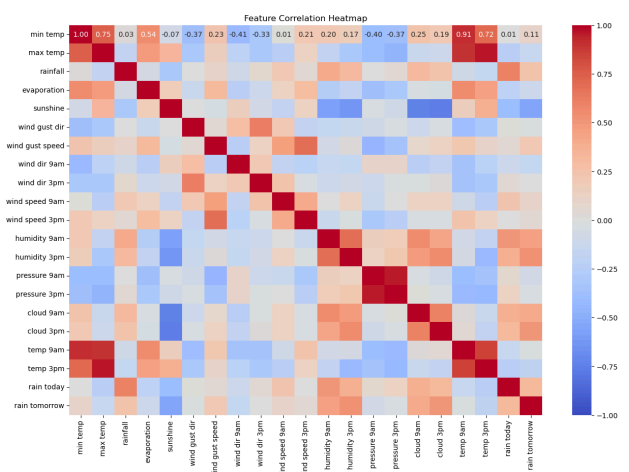


Figure 17 : Feature Correlation Heatmap

4.4.1.2 Filtering Data

This analysis aims to analyse what factors would affect rain probability if it trained today. Although whether it rained today isn't the sole influence on whether it will rain tomorrow, and factors contribute to the probability of rain the next day, it has a significant impact, and so the data was filtered to remove all samples when it didn't rain today.

4.4.2 Results

4.2.2.1 Combination of Features that Correlate with the Predictor

To identify the optimal feature set, an Automatic Forward Selection algorithm was employed. The process began with a model that included no variables. The feature demonstrating the highest correlation with the predictor was then chosen and incorporated into the model. If this addition enhanced the model's validation accuracy by at least 0.005, the algorithm proceeded to select the next feature. This iterative approach continued until no further improvements met the specified threshold.

4.2.2.2 Confusion Matrix

By plotting a confusion matrix before analysing the data provided immediate feedback on how well the model classified the data, highlighting patterns of correct and incorrect predictions. This visual assessment helps identify areas where the model may struggle. By establishing a baseline performance early on, decisions on further analysis were guided, model tuning, and potential adjustments to data preprocessing or feature selection strategies.

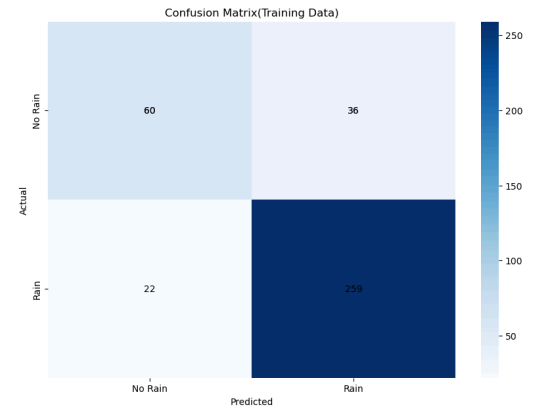


Figure 18: Confusion Matrix for Training Data

Based on the confusion matrix, the overall accuracy, can be calculated as $\frac{TP+TN}{TP+TN+FP+FN}$. In this case, accuracy would be $\frac{259+60}{259+60+36+22} = \frac{319}{377} \approx 0.846$ or 84.6%.

Additionally, precision and recall were calculated to identify the proportion of positive predictions (predicted rain) that were actually correct and to measure the proportion of actual positives (rainy days) that were correctly identified by the model, respectively. Precision is calculated as $\frac{TP}{TP+FP} = \frac{259}{259+36} \approx 0.878$ or 87.7%. Recall is calculated as $\frac{TP}{TP+FN} = \frac{259}{259+22} \approx 0.921$, or 92.1%.

Overall, the model appears reasonably effective in predicting rainy days (positive class), with high True Positive rate (Recall) suggesting it captures a significant portion of actual rainy days. Addressing False Negatives (FN) could be a priority if missing rainy days is costly or undesirable, possibly through adjusting model thresholds or improving feature selection.

4.2.2.2 Validation

To avoid overfitting, the models from the forward selection process were saved, and the accuracy produced when applying the test set to them was analysed.

The figure below show the relationship between the number of features and the average model accuracy.

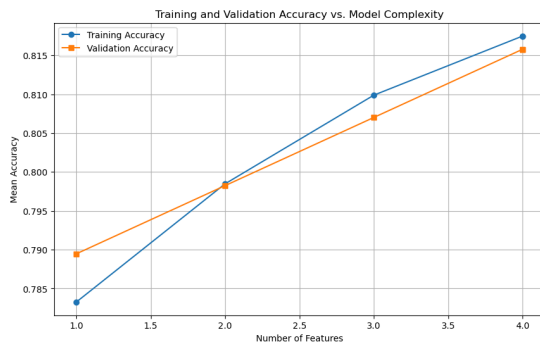


Figure 19: Model Complexity vs Model Accuracy

4.2.2.3 Final Results

	Accuracy	Precision	Recall
Train 1	0.814	0.848	0.903
Train 2	0.767	0.818	0.871
Validation 1	0.767	0.818	0.871
Validation 2	0.791	0.824	0.903

Table 5: Results of Logistic Regression Forward Selection Model

4.4.3 Discussion

Logistic regression forward selection conveys the factors that most affect next day rain prediction for days when it rained the previous day. These most important features are: humidity 3pm, wind dir 3pm, wind gust dir, max temp, temp 9am, temp 3pm and cloud 3pm.

As these features were added, the validation accuracy decreased, from initially 0.85 to 0.8. It was observed how some features had very similar validation accuracy improvement than others, suggesting both features are effective at predicting the target variable and can outline

some correlation among features, as they might be capturing similar underlying patterns in the data or might have redundant information (for instance 'wind dir 3pm' and 'wind gust dir' are very probable to be overlapping data).

As this is a weather dataset for a very sensitive to meteorological conditions area (Sydney airport), the model should have a high sensitivity so that it can predict the probability of rain the next day. This is not only because \$3.35 billion dollars a year are allocated to airport IT improvement [6], but also because passenger's and airline's security and schedule rely on efficiency and punctuality of flights. It is better to have a more positive diagnosis than negative ones, as this reduces the need to cancel or delay flights.

For both the validation and training sets, the value for accuracy, precision and recall were all high, however, some overfitting was started to become visible at model complexity 4. The model had a high recall of 0.903 which results in high sensitivity.

To further improve the logistic regression model, conducting both backward and forward feature selection could be highly beneficial. Forward selection involves starting with no features and adding them one by one, based on their contribution to the model's performance, which can help identify the most impactful features. Conversely, backward selection starts with all potential features and systematically removes the least significant ones, allowing the model to eliminate noise and reduce overfitting. By combining these approaches, the model could be optimised to include only the most relevant features, potentially enhancing overall performance, including recall, and achieving a more robust and generalizable predictive model. Additionally, incorporating more training data, particularly examples of positive raining days, can enhance the model's learning as well as regularisation techniques to prevent overfitting,

and adjusting the decision threshold to balance precision and recall more effectively.

5. Conclusion

All the models tested exhibited an accuracy between 0.67 and 0.79, for the testing data, suggesting that the possibility of rain the next day generally can be reliably predicted based upon the selected features.

It is possible to conclude that logistic regression is the most suitable model, with a validation accuracy of 79%, as the question addressed remained the same for every model, and the data pre-processing took place within the group, using the same training, validation and testing data.

A heatmap (more detailed version attached in appendix) was calculated for the data.

By doing this, the strength and direction of relationships between different variables can be easily identified. In future analysis, by observing the correlations, redundant features highly correlated with each other could be removed or combined to simplify the model without losing important information and thus improving model performance. Additionally, by focusing on agreed most relevant features, a more accurate and efficient model could be built.

Another factor that would improve the accuracy of all the models would be reducing outliers during the pre-processing of the data, as the final dataset we were left with had considerably fewer samples compared to the initial one. The smaller the dataset, the more of an impact the outliers will have on the mean and spread of the data.

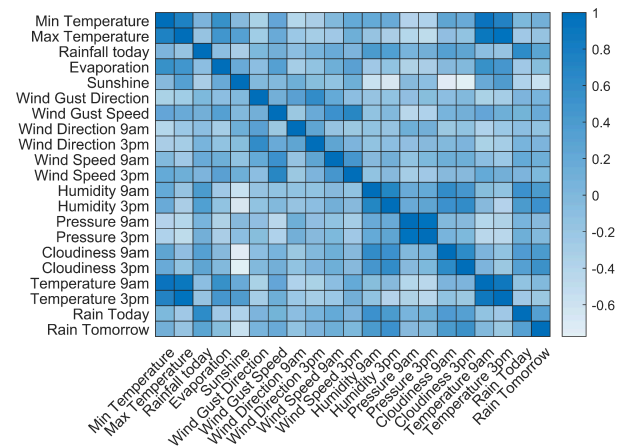
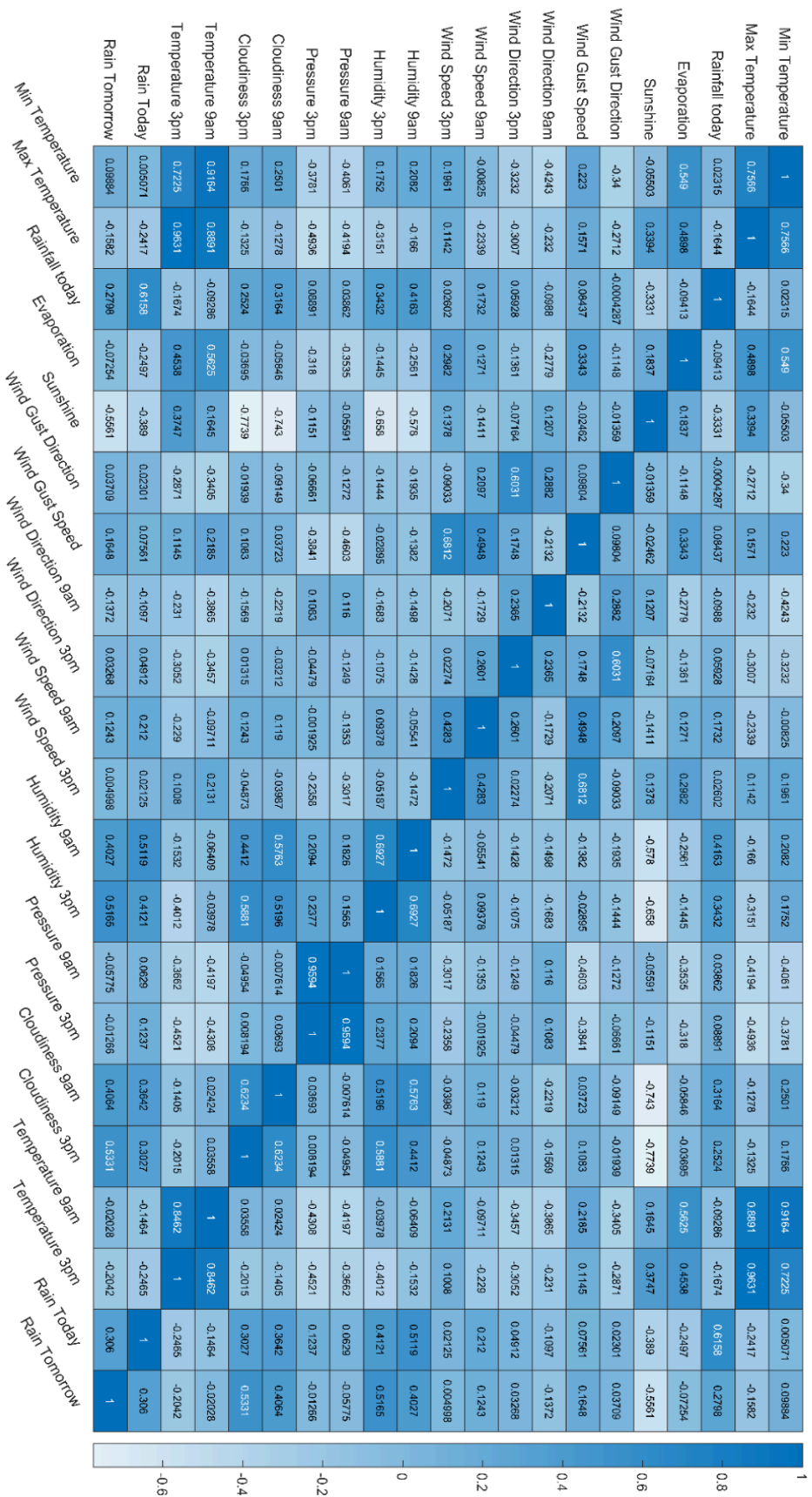
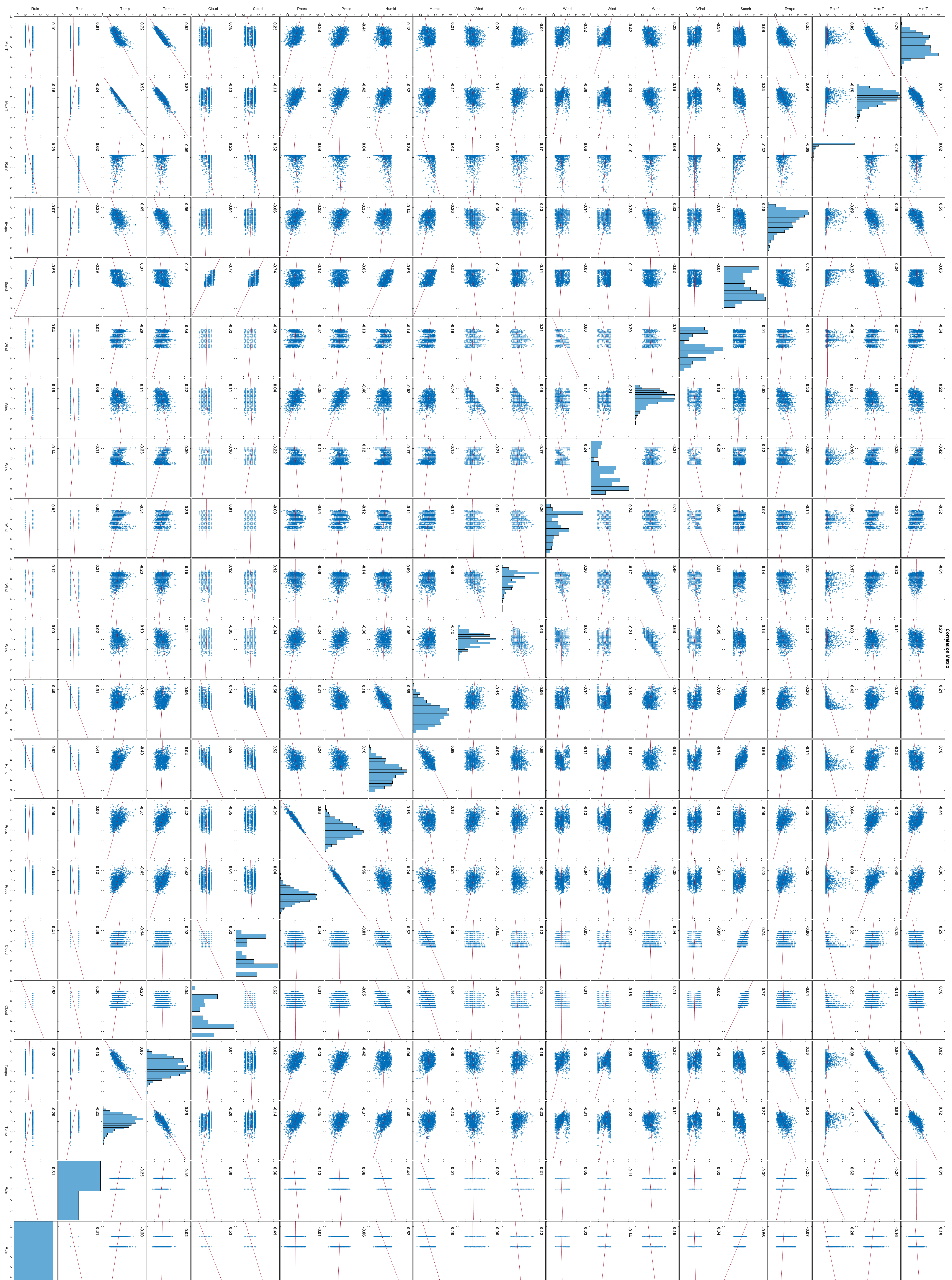


Figure 20: Heatmap of data

A correlation plot was also drawn (Showing the data distributions for each (pre-processed) feature and all possible 2 feature combinations with a linear regression line of best plotted) is also attached in the appendix.

6. Appendix





6.1 Preprocessing data

Excel formulae to convert compass rose values to headings:

```
=IF(I1729="N",0,  
(IF(I1729="NNE",22.5,(IF(I1729="NE",45,(IF(I1729="ENE",67.5,(IF(I1729="E",90,(IF(I1729="ESE",112.  
5,(IF(I1729="SE",135,(IF(I1729="SSE",157.5,(IF(I1729="S",180,(IF(I1729="SSW",202.5,(IF(I1729="SW  
",225,(IF(I1729="WSW",247.5,(IF(I1729="W",270,(IF(I1729="WNW",292.5,(IF(I1729="NW",315,(IF(I17  
29="NNW",337.5,"Err"))))))))))))))))))))))))))))))))))))))))))))))))))))))))))))))))
```

Matlab normalisation, standardisation and splitting code:

```
data = csvread("sydney_airpor_rain_data_clean_numerical.csv");  
numRows = size(data, 1)  
nonSkewedData = [];  
nonSkewedData = data(data(:,21)==1,:);  
numYes = size(nonSkewedData,1)  
i=1;  
while i < numYes  
    currentRow = data(round(numRows*rand()),:);  
    if currentRow(21) == 0  
        nonSkewedData = [nonSkewedData; currentRow];  
        i = i + 1;  
    end  
end  
normalizedData = normalize(nonSkewedData(:,1:19));  
normalizedData = horzcat(normalizedData, nonSkewedData(:,20:21));  
newOrder = randperm(size(normalizedData,1));  
shuffledNormalisedData = normalizedData;  
shuffledNormalisedData = normalizedData(newOrder,:);  
csvwrite("clean_normalized_data.csv", shuffledNormalisedData);  
% autosplit function (didnt work, yet)  
% ratios = [8,2,10,14];  
% startIndex = 1;  
% endIndex = fix((ratios(1)/sum(ratios))*size(shuffledNormalisedData,1))  
% for c = 2:1:size(ratios,2)  
%     csvwrite("clean_normalized_split_" + string(c) + "_data.csv",  
shuffledNormalisedData(startIndex:endIndex,:));  
%     startIndex = endIndex + 1  
%     endIndex = startIndex +  
fix((ratios(c)/sum(ratios))*size(shuffledNormalisedData,1))  
% end  
startIndex=1  
endIndex = fix(0.8*size(shuffledNormalisedData,1))  
csvwrite("clean_normalized_split_" + string(1) + "_data.csv",  
shuffledNormalisedData(startIndex:endIndex,:));  
startIndex = endIndex + 1  
endIndex = startIndex + fix(0.1*size(shuffledNormalisedData,1))
```

```

csvwrite("clean_normalized_split_" + string(2) + "_data.csv",
shuffledNormalisedData(startIndex:endIndex,:));
startIndex = endIndex + 1
endIndex = startIndex + fix(0.1*size(shuffledNormalisedData,1))
csvwrite("clean_normalized_split_" + string(3) + "_data.csv",
shuffledNormalisedData(startIndex:endIndex,:));

```

6.2 Decision Trees

importing required libraries

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_graphviz
from sklearn.metrics import accuracy_score, precision_score, recall_score,
confusion_matrix
import graphviz
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

```

```

training = pd.read_csv('training_data.csv', header=None)
validation = pd.read_csv('validation_data.csv', header=None)
testing = pd.read_csv('testing_data.csv', header=None)

```

```

print(training.head())
print(validation.head())
print(testing.head())

```

assigning variables to columns?

```

training_y = training.iloc[:, -1]
training_x = training.iloc[:, :-1]

validation_y = validation.iloc[:, -1]
validation_x = validation.iloc[:, :-1]

testing_y = testing.iloc[:, -1]
testing_x = testing.iloc[:, :-1]

```

optimising max_depth

```

max_depth_values = range(1, 15)
training_accuracies = []
validation_accuracies = []

for max_depth in max_depth_values:
    clf = DecisionTreeClassifier(criterion='gini', max_depth=max_depth, random_state=0)

    clf.fit(training_x, training_y)

    training_y_pred = clf.predict(training_x)
    training_accuracy = accuracy_score(training_y, training_y_pred)
    training_accuracies.append(training_accuracy)

    validation_y_pred = clf.predict(validation_x)
    validation_accuracy = accuracy_score(validation_y, validation_y_pred)
    validation_accuracies.append(validation_accuracy)

```

```

# Plot

plt.figure(figsize=(10, 6))
plt.plot(max_depth_values, training_accuracies, label='Training Accuracy', marker='o')
plt.plot(max_depth_values, validation_accuracies, label='Validation Accuracy',
marker='o')
plt.xlabel('Max Depth')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy vs Max Depth')
plt.legend()
plt.grid(True)
plt.show()

```

optimising min_impurity

```

min_impurity_values = np.arange(0.0, 0.05, 0.01)

training_accuracies = []
validation_accuracies = []

for min_impurity in min_impurity_values:
    clf = DecisionTreeClassifier(criterion='gini', min_impurity_decrease=min_impurity,
random_state=0)

    clf.fit(training_x, training_y)

    training_y_pred = clf.predict(training_x)

    training_accuracy = accuracy_score(training_y, training_y_pred)
    training_accuracies.append(training_accuracy)

    validation_y_pred = clf.predict(validation_x)

    validation_accuracy = accuracy_score(validation_y, validation_y_pred)
    validation_accuracies.append(validation_accuracy)

plt.figure(figsize=(10, 6))
plt.plot(min_impurity_values, validation_accuracies, marker='o', label='Validation
Accuracy')
plt.plot(min_impurity_values, training_accuracies, marker='o', label='Testing Accuracy')
plt.title('Accuracy vs. Max Depth')
plt.xlabel('Max Depth')
plt.ylabel('Accuracy')
plt.xticks(min_impurity_values)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Calculating accuracy, precision and recall of training and validation data

```

clf = DecisionTreeClassifier(criterion='gini', max_depth=3, min_impurity_decrease=0.01,
random_state=0)
clf.fit(training_x, training_y)

# decision tree creation

```

```

tree_data = export_graphviz(clf, out_file=None,
                             feature_names=[f'Feature {i}' for i in
range(training_x.shape[1])],
                             class_names=[str(cls) for cls in set(training_y)],
                             filled=True, rounded=True,
                             special_characters=True)

graph = graphviz.Source(tree_data)

graph.render("decision_tree")
graph.view()

training_accuracy = accuracy_score(training_y, training_y_pred)
print(f'Accuracy on the training set: {training_accuracy}')
```



```

validation_accuracy = accuracy_score(validation_y, validation_y_pred)
print(f'Accuracy on the validation set: {validation_accuracy}')
```



```

training_precision = precision_score(training_y, training_y_pred, average='weighted')
training_recall = recall_score(training_y, training_y_pred, average='weighted')

print(f'Training Precision: {training_precision:.4f}')
```



```

print(f'Training Recall: {training_recall:.4f}')
```



```

validation_precision = precision_score(validation_y, validation_y_pred,
average='weighted')
validation_recall = recall_score(validation_y, validation_y_pred, average='weighted')

print(f'Validation Precision: {validation_precision:.4f}')
```



```

print(f'Validation Recall: {validation_recall:.4f}')
```



```

testing_y_pred = clf.predict(testing_x)

testing_accuracy = accuracy_score(testing_y, testing_y_pred)
print(f'Accuracy on the testing set: {testing_accuracy}')
```



```

testing_precision = precision_score(testing_y, testing_y_pred, average='weighted')
testing_recall = recall_score(testing_y, testing_y_pred, average='weighted')

print(f'testing Precision: {testing_precision:.4f}')
```



```

print(f'testing Recall: {testing_recall:.4f}')
```


Creating confusion matrix


```

training_confusion_matrix = confusion_matrix(training_y, training_y_pred)
validation_confusion_matrix = confusion_matrix(validation_y, validation_y_pred)

print(f'Training Confusion Matrix:\n{training_confusion_matrix}')
```



```

print(f'Validation Confusion Matrix:\n{validation_confusion_matrix}')
```



```

#seaborn plot?
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

sns.heatmap(training_confusion_matrix, annot=True, fmt='d', ax=ax[0], cmap='Blues')
ax[0].set_title('Training Confusion Matrix')
ax[0].set_xlabel('Predicted')
ax[0].set_ylabel('Actual')
```

```

sns.heatmap(validation_confusion_matrix, annot=True, fmt='d', ax=ax[1], cmap='Blues')
ax[1].set_title('Validation Confusion Matrix')
ax[1].set_xlabel('Predicted')
ax[1].set_ylabel('Actual')

plt.show()

```

6.3 SVM

```

trainingData = csvread("clean_normalized_split_1_data.csv");
validationData = csvread("clean_normalized_split_2_data.csv");
testingData = csvread("clean_normalized_split_3_data.csv");
%the validation partition is extracted automatically from training data
trainingData = vertcat(trainingData, validationData);

myfun = @(XTrain,yTrain,XTest,yTest) ...
    size(XTest,1)*loss(fitcsvm(XTrain,yTrain),XTest,yTest);
cv = cvpartition(trainingData(:,21:21),"KFold",10);
opts = statset("Display","iter",UseParallel=true);
tf = sequentialfs(myfun,trainingData(:,1:20),trainingData(:,21:21),"CV",cv,"Options",opts);

opts = struct('CVPartition',cv,'AcquisitionFunctionName', ...
    'expected-improvement-plus', 'UseParallel',true);
SVMModel = fitcsvm(trainingData(:,tf), trainingData(:,21:21),'Standardize', 'off', ...
    'OptimizeHyperparameters','auto','HyperparameterOptimizationOptions',opts)
[predictions, score] = predict(SVMModel,testingData(:,tf));
loss(SVMModel, trainingData(:,tf), trainingData(:,21:21))
disp([predictions testingData(:,21:21)]);
confusionchart(confusionmat(predictions, testingData(:,21:21)), {'yes', 'no'})

```

6.4 Random Forest

6.4.1 Data Sorting and Confusion Matrix

```

import pandas as pd

# Read CSV

dataset = pd.read_csv(r'C:\Users\tcarp\Documents\Data
Science\sydney_airport_unsplit.csv')

print(dataset.head())

print(dataset.info())

#Seaborn Heatmap to identify which variable comumns to drop

```

```

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,
confusion_matrix

# Calculate the correlation matrix
corr_matrix = dataset.corr()

# Create a heatmap
plt.figure(figsize=(14, 8))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm')
plt.show()

print(dataset.columns)

#remove redundancys in dataset
dataset = dataset.drop(['Min Temp', 'Pressure9am','WindDir9am', 'WindSpeed9am',
'Humidity9am', 'Temp9am','Cloud9am' ], axis=1)

print(dataset.columns)

# Balancing dataset
ratings_count = dataset['RainTomorrow'].value_counts()
min_sample_number = ratings_count.min()
print("Minimum sample size per class:", min_sample_number)

# Select features and target
X = dataset.drop('RainTomorrow', axis=1) # drop the target column
y = dataset['RainTomorrow'].astype('int') # target column

# Split the data into training, validation, and test sets
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.2,
random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5,
random_state=42)

# Initialize the Random Forest model
model = RandomForestClassifier(n_estimators=100, max_depth=None,
min_samples_split=2, min_impurity_decrease=0.01, random_state=0)
model.fit(X_train, y_train)

```

```
#testing validation and training result

def evaluate_model(model, X, y, name="Dataset"):
    y_pred = model.predict(X)
    acc = accuracy_score(y, y_pred)
    prec = precision_score(y, y_pred, average='macro', zero_division=0)
    rec = recall_score(y, y_pred, average='macro', zero_division=0)
    print(f"{name} - Accuracy: {acc}, Precision: {prec}, Recall: {rec}")

evaluate_model(model, X_train, y_train, "Training")
evaluate_model(model, X_val, y_val, "Validation")
evaluate_model(model, X_test, y_test, "Test")
```

6.4.2 model and hyperparameter optimisation

```
# Tuning max_depth hyperparameter and plotting the accuracies
acc_train = []
acc_val = []

for depth in range(1, 27):
    dt = RandomForestClassifier(max_depth=depth, min_samples_split=0.1,
random_state=0, min_impurity_decrease=0.01)
    dt.fit(X_train, y_train)
    ypred_train = dt.predict(X_train)
    ypred_val = dt.predict(X_val)
    acc_train.append(accuracy_score(y_train, ypred_train))
    acc_val.append(accuracy_score(y_val, ypred_val))

plt.figure(figsize=(10, 6))
plt.plot(range(1, 27), acc_train, 'bo-', label='Training Data')
plt.plot(range(1, 27), acc_val, 'ro-', label='Validation Data')
plt.xlabel("Max Depth of Trees")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy vs. Max Depth")
plt.legend()
plt.grid(True)
plt.show()
```

```
# Initialize the Random Forest model
model = RandomForestClassifier(n_estimators=100, max_depth=3,
min_samples_split=2, min_impurity_decrease=0.01, random_state=0)
```

```

model.fit(X_train, y_train)
# Array to store accuracy for training and validation sets
acc_train = []
acc_val = []

# Define the range for min_samples_split
samples = np.linspace(0.01, 1, 10, endpoint=True)

# Iterating over the range of min_samples_split values
for i in samples:
    # Initialize and fit the RandomForest model on the training data
    dt = RandomForestClassifier(max_depth=3, min_samples_split=i,
random_state=0)
    dt.fit(X_train, y_train)

    # Make predictions with the trained model
    ypred_train = dt.predict(X_train)
    ypred_val = dt.predict(X_val)

    # Calculate accuracy and append to lists
    acc_train.append(accuracy_score(y_train, ypred_train))
    acc_val.append(accuracy_score(y_val, ypred_val))

# Plotting the accuracies to visualize the effect of min_samples_split on model
performance
plt.figure(figsize=(10, 6))
plt.plot(samples, acc_train, 'bo-', label='Training Data')
plt.plot(samples, acc_val, 'ro-', label='Validation Data')
plt.xlabel("Min Samples Split")
plt.ylabel("Accuracy")
plt.title("Effect of Min Samples Split on Training and Validation Accuracy")
plt.legend()
plt.grid(True)
plt.show()

# Initialize the Random Forest model
model = RandomForestClassifier(n_estimators=100, max_depth=3,
min_samples_split=0.1, min_impurity_decrease=0.01, random_state=0)
model.fit(X_train, y_train)

# Arrays to store accuracy for training and validation sets
acc_train = []
acc_val = []

# Define the range for min_impurity_decrease

```

```

samples = np.linspace(0.01, 1, 10, endpoint=True)

# Iterating over the range of min_impurity_decrease values
for impurity in samples:
    # Initialize and fit the RandomForest model on the training data
    dt = RandomForestClassifier(max_depth=3, min_samples_split=0.1,
random_state=0,
                                min_impurity_decrease=impurity)

    dt.fit(X_train, y_train)

    # Make predictions with the trained model
    ypred_train = dt.predict(X_train)
    ypred_val = dt.predict(X_val)

    # Calculate accuracy and append to lists
    acc_train.append(accuracy_score(y_train, ypred_train))
    acc_val.append(accuracy_score(y_val, ypred_val))

# Plotting the accuracies to visualize the effect of min_impurity_decrease on
model performance
plt.figure(figsize=(10, 6))
plt.plot(samples, acc_train, 'bo-', label='Training Data')
plt.plot(samples, acc_val, 'ro-', label='Validation Data')
plt.xlabel("Min Impurity Decrease")
plt.ylabel("Accuracy")
plt.title("Effect of Min Impurity Decrease on Training and Validation
Accuracy")
plt.legend()
plt.grid(True)
plt.show()

# Initialize the Random Forest model
model = RandomForestClassifier(n_estimators=100, max_depth=3,
min_samples_split=0.1, min_impurity_decrease=0, random_state=0)
model.fit(X_train, y_train)

# Iterating over a range of n_estimators values
for n in range(1, 100, 4):
    # Initialize and fit the RandomForest model on the training data
    dt = RandomForestClassifier(max_depth=3, min_samples_split=0.1,
random_state=0,
                                min_impurity_decrease=0, n_estimators=n)

    dt.fit(X_train, y_train)

    # Make predictions with the trained model

```

```

ypred_train = dt.predict(X_train)
ypred_val = dt.predict(X_val)

# Calculate accuracy and append to lists
acc_train.append(accuracy_score(y_train, ypred_train))
acc_val.append(accuracy_score(y_val, ypred_val))

# Plotting the accuracies to visualize the effect of n_estimators on model
performance
plt.figure(figsize=(10, 6))
plt.plot(range(1, 100, 4), acc_train, 'bo-', label='Training Data')
plt.plot(range(1, 100, 4), acc_val, 'ro-', label='Validation Data')
plt.xlabel("Number of Estimators")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy vs. Number of Estimators")
plt.legend()
plt.grid(True)
plt.show()

# Initialize the Random Forest model
model = RandomForestClassifier(n_estimators=80, max_depth=3,
min_samples_split=0.1, min_impurity_decrease=0, random_state=0)
model.fit(X_train, y_train)

import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score

# Helper function to print evaluation metrics
def print_evaluation_metrics(y_true, y_pred, dataset_name=""):
    print(f"{dataset_name} Metrics:")
    print(f"Accuracy: {accuracy_score(y_true, y_pred):.3f}")
    print(f"Precision: {precision_score(y_true, y_pred, average='macro',
zero_division=0):.3f}")
    print(f"Recall: {recall_score(y_true, y_pred, average='macro',
zero_division=0):.3f}\n")

# Model 1 - Tuning Max Depth
print("Model 1 - Tuning Max Depth")
for depth in range(1, 30):
    model = RandomForestClassifier(max_depth=depth, n_estimators=100,
min_samples_split=2, min_impurity_decrease=0.01, random_state=0)
    model.fit(X_train, y_train)
    ypred_train = model.predict(X_train)

```

```

ypred_val = model.predict(X_val)
ypred_test = model.predict(X_test)
print(f"Depth: {depth}")
print_evaluation_metrics(y_train, ypred_train, "Training")
print_evaluation_metrics(y_val, ypred_val, "Validation")
print_evaluation_metrics(y_test, ypred_test, "Test")

# Model 2 - Tuning Min Samples Split
print("Model 2 - Tuning Min Samples Split")
samples_split_range = np.linspace(0.01, 1, 10)
for split in samples_split_range:
    model = RandomForestClassifier(max_depth=3, min_samples_split=split,
random_state=0)
    model.fit(X_train, y_train)
    ypred_train = model.predict(X_train)
    ypred_val = model.predict(X_val)
    ypred_test = model.predict(X_test)
    print(f"Min Samples Split: {split:.2f}")
    print_evaluation_metrics(y_train, ypred_train, "Training")
    print_evaluation_metrics(y_val, ypred_val, "Validation")
    print_evaluation_metrics(y_test, ypred_test, "Test")

# Model 3 - Tuning Min Impurity Decrease
print("Model 3 - Tuning Min Impurity Decrease")
impurity_decrease_range = np.linspace(0.01, 1, 10)
for impurity in impurity_decrease_range:
    model = RandomForestClassifier(max_depth=3, min_samples_split=0.1,
min_impurity_decrease=impurity, random_state=0)
    model.fit(X_train, y_train)
    ypred_train = model.predict(X_train)
    ypred_val = model.predict(X_val)
    ypred_test = model.predict(X_test)
    print(f"Min Impurity Decrease: {impurity:.2f}")
    print_evaluation_metrics(y_train, ypred_train, "Training")
    print_evaluation_metrics(y_val, ypred_val, "Validation")
    print_evaluation_metrics(y_test, ypred_test, "Test")

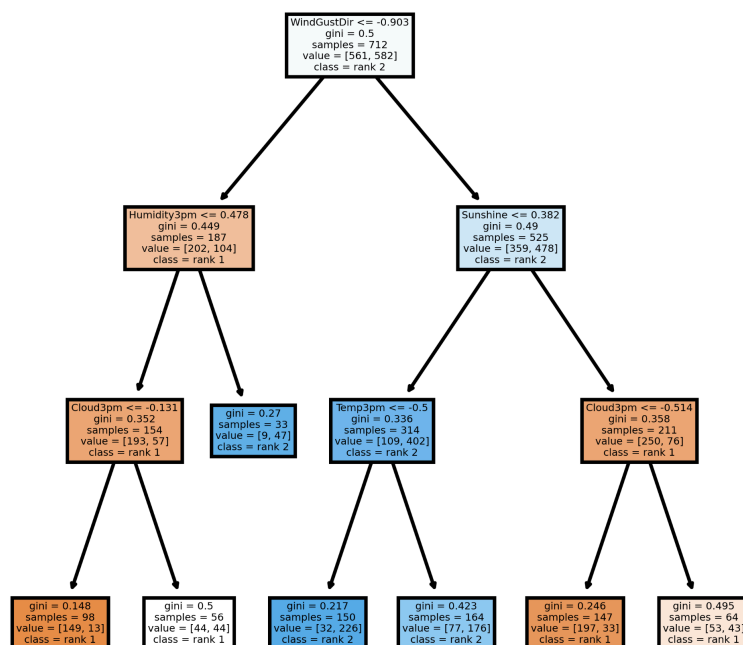
# Model 4 - Testing N Estimators
print("Model 4 - Testing N Estimators")
estimators_range = range(1, 100, 10)
for n_estimators in estimators_range:
    model = RandomForestClassifier(max_depth=7, min_samples_split=0.1,
min_impurity_decrease=0, n_estimators=n_estimators, random_state=0)

```

```

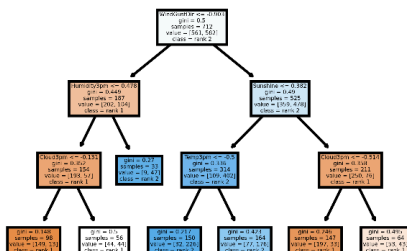
model.fit(X_train, y_train)
ypred_train = model.predict(X_train)
ypred_val = model.predict(X_val)
ypred_test = model.predict(X_test)
print(f"N Estimators: {n_estimators}")
print_evaluation_metrics(y_train, ypred_train, "Training")
print_evaluation_metrics(y_val, ypred_val, "Validation")
print_evaluation_metrics(y_test, ypred_test, "Test")

```

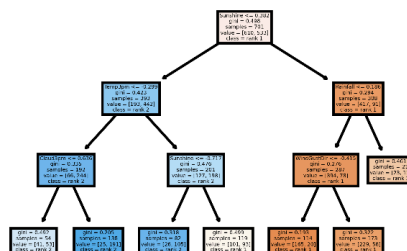


6.4.3 plot tree

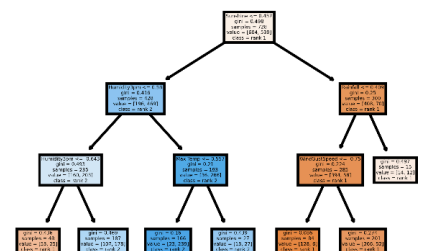
Estimator: 0



Estimator: 1



Estimator: 2



```

from sklearn import tree
import matplotlib.pyplot as plt

```

```

# Display a single final tree
fig, ax = plt.subplots(figsize=(4, 4), dpi=800)
tree.plot_tree(model.estimators_[0], feature_names=X_train.columns,
class_names=["rank 1", "rank 2", "rank 3", "rank 4"], filled=True, ax=ax)
fig.savefig("rf_individualtree.png")
plt.show()

# Display trees of 3 estimators
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(10, 2), dpi=900)
for index in range(3): # Corrected to display 3 trees as indicated
    tree.plot_tree(model.estimators_[index], feature_names=X_train.columns,
class_names=["rank 1", "rank 2", "rank 3", "rank 4"], filled=True, ax=axes[index])
    axes[index].set_title(f"Estimator: {index}", fontsize=11)
fig.savefig("rf_3_trees.png")
plt.show()

import pandas as pd
import numpy as np

# Calculating feature importances and their standard deviation
importances = model.feature_importances_
std = np.std([tree.feature_importances_ for tree in model.estimators_], axis=0)
forest_importances = pd.Series(importances, index=X_train.columns)

# Plotting feature importances
fig, ax = plt.subplots()
forest_importances.plot.bar(yerr=std, ax=ax)
ax.set_title("Feature Importances using MDI")
ax.set_ylabel("Mean Decrease in Impurity")
fig.tight_layout()
fig.savefig("feature_importance.png")
plt.show()

```

6.4.4 Confusion matrix plot and GridSearch Cv code

```

# Assuming model has been trained and X_val, y_val are available
# Predict on validation set
y_pred_val = model.predict(X_val)

# Generate the confusion matrix
cm = confusion_matrix(y_val, y_pred_val)

# Plotting the confusion matrix
fig, ax = plt.subplots(figsize=(8, 8)) # Set appropriate size for the plot

```

```

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax) # 'fmt' is string
formatting code for decimal integers

# Labels, title and ticks
label_names = ['No Rain', 'Rain'] # Change labels appropriately
ax.set_xlabel('Predicted labels')
ax.set_ylabel('True labels')
ax.set_title('Confusion Matrix')
ax.xaxis.set_ticklabels(label_names)
ax.yaxis.set_ticklabels(label_names)
plt.show()

#Automated Hyperparameter Tuning Using GridSearchCv

from sklearn.model_selection import GridSearchCV

# Define the parameter grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Initialize the GridSearchCV object
grid_search = GridSearchCV(estimator=RandomForestClassifier(random_state=0),
param_grid=param_grid, cv=3, verbose=2, n_jobs=-1)

# Fit GridSearchCV
grid_search.fit(X_train, y_train)

# Best parameters and best score
print("Best parameters:", grid_search.best_params_)
print("Best cross-validation score: {:.2f}".format(grid_search.best_score_))

# Use the best estimator to make predictions
model_best = grid_search.best_estimator_
y_val_pred = model_best.predict(X_val)
print("Validation Accuracy with Best Parameters:", accuracy_score(y_val,
y_val_pred))

```

6.5 Logistic Regression

```
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression as logreg
from scipy import stats
import warnings
warnings.filterwarnings('ignore') # prevents future version warnings
from sklearn.metrics import confusion_matrix, accuracy_score
```

6.4.1 Data Sorting and Confusion Matrix

```
training_data = pd.read_csv('training_data.csv', delimiter=';', header = 0)
training_data_rain_today= training_data.drop(training_data[training_data['rain today'] == 0].index)
training_data_rain_today.head()
```

```
X_train = training_data_rain_today[["min temp", "max temp", "rainfall", "evaporation", "sunshine", "wind gust dir", "wind gust speed", "wind dir 9am", "wind dir 3pm", "wind speed 9am", "wind speed 3pm", "humidity 9am", "humidity 3pm", "pressure 9am", "pressure 3pm", "cloud 9am", "cloud 3pm", "temp 9am", "temp 3pm", "rain today"]]
y_train = training_data_rain_today["rain tomorrow"]
```

```
mylr = logreg(C=1e9).fit(X_train,y_train)
```

```
model_summary = ModelSummary(mylr,X_train,y_train)
model_summary.get_summary()
```

```
cm = confusion_matrix(y_train, y_pred)
```

```
# Plot the confusion matrix
```

```
plt.figure(figsize=(10, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No Rain', 'Rain'], yticklabels=['No Rain', 'Rain'])
```

```
for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(j + 0.5, i + 0.5, cm[i, j], ha='center', va='center', color='black')
```

```
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix(Training Data)')
plt.show()
```

6.4.2 Plotting Correlation Heatmap

```
# Selecting the features for correlation
```

```
features = ["min temp", "max temp", "rainfall", "evaporation", "sunshine", "wind gust dir",
            "wind gust speed", "wind dir 9am", "wind dir 3pm", "wind speed 9am",
            "wind speed 3pm", "humidity 9am", "humidity 3pm", "pressure 9am",
            "pressure 3pm", "cloud 9am", "cloud 3pm", "temp 9am", "temp 3pm", "rain today"]
```

```
# Computing the correlation matrix
```

```
correlation_matrix = training_data[features].corr()
```

```
# Plotting the heatmap
plt.figure(figsize=(15, 10))
sns.heatmap(correlation_matrix, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1)

# Adding title and labels
plt.title('Feature Correlation Heatmap')
plt.show()
```

6.4.2 Logistic Regression

```
def select_column_to_add(X_train, y_train, X_val, y_val, columns_in_model, columns_to_test):
    column_best = None
    columns_in_model = list(columns_in_model)

    if len(columns_in_model) == 0:
        acc_best = 0
    elif len(columns_in_model) == 1:
        mylr = logreg(C=1e9).fit(X_train[columns_in_model].values.reshape(-1, 1), y_train)
        acc_best = accuracy_score(y_val, mylr.predict(X_val[columns_in_model].values.reshape(-1, 1)))
    else:
        mylr = logreg(C=1e9).fit(X_train[columns_in_model], y_train)
        acc_best = accuracy_score(y_val, mylr.predict(X_val[columns_in_model]))

    for column in columns_to_test:
        mylr = logreg(C=1e9).fit(X_train[columns_in_model + [column]], y_train)
        y_pred = mylr.predict(X_val[columns_in_model + [column]])
        acc = accuracy_score(y_val, y_pred)

        if acc - acc_best >= 0.005:
            acc_best = acc
            column_best = column

    if column_best is not None:
        print("Adding {} to the model".format(column_best))
        print("The new best validation accuracy is {}".format(acc_best))
        columns_in_model_updated = columns_in_model + [column_best]
    else:
        print("Nothing was added to the model")
        columns_in_model_updated = columns_in_model

    return (columns_in_model_updated, acc_best)
```

6.4.3 Forward Selection

```
def auto_forward_selection(X_train, y_train, X_val, y_val, max_num_of_features):
    columns_to_test = list(X_train.columns)
    columns_in_model = []
    current_acc = 0

    for i in range(max_num_of_features):
        columns_in_model, acc_best = select_column_to_add(X_train, y_train, X_val, y_val, columns_in_model,
            columns_to_test)
        if acc_best == current_acc:
```

```

        break
    else:
        current_acc = acc_best
        columns_to_test = [feature for feature in columns_to_test if feature not in columns_in_model]
        print(columns_in_model, acc_best)
    return columns_in_model, acc_best

```

#running function multiple times with different features

```

col_in_m1,acc_best1 = auto_forward_selection(X_train,y_train,X_val,y_val,5)
col_in_m2,acc_best2 = auto_forward_selection(X_train.drop(columns = ['max temp']),y_train,X_val.drop(columns
= ['max temp']),y_val,5)
col_in_m3,acc_best3 = auto_forward_selection(X_train.drop(columns = ['max temp','wind dir
3pm']),y_train,X_val.drop(columns = ['max temp', 'wind dir 3pm']),y_val,5)
col_in_m4,acc_best4 = auto_forward_selection(X_train.drop(columns = ['max temp','wind gust dir', 'temp
9am']),y_train,X_val.drop(columns = ['max temp','wind gust dir', 'temp 9am']),y_val,5)
col_in_m5,acc_best5 = auto_forward_selection(X_train.drop(columns = ['max temp','wind gust dir', 'temp
9am','temp 3pm']),y_train,X_val.drop(columns = ['max temp','wind gust dir', 'temp 9am','temp 3pm']),y_val,5)

```

6.4.4 Finding Test Accuracies

```

from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
from sklearn.linear_model import LogisticRegression as logreg

```

#Standardization of data

```

means = X_train.mean(axis=0)
stds = X_train.mean(axis=0)

```

```

X_test_stand = (X_test-means)/stds
X_train_stand = (X_train-means)/stds

```

#Subset selection

#Selects a subset of standardized training data based on different previously determined columns

```

X1_train = X_train_stand[col_in_m1]
X2_train = X_train_stand[col_in_m2]
X3_train = X_train_stand[col_in_m3]
X4_train = X_train_stand[col_in_m4]
X5_train = X_train_stand[col_in_m5]

```

#Model Training

#Initializes and fits a logistic regression model on different subsets with the corresponding labels

```

mylr1 = logreg().fit(X1_train,y_train)
mylr2 = logreg().fit(X2_train,y_train)
mylr3 = logreg().fit(X3_train,y_train)
mylr4 = logreg().fit(X4_train,y_train)
mylr5 = logreg().fit(X5_train,y_train)

```

#Prediction & Evaluation

```

X1_test = X_test_stand[col_in_m1]
X2_test = X_test_stand[col_in_m2]
X3_test = X_test_stand[col_in_m3]
X4_test = X_test_stand[col_in_m4]
X5_test = X_test_stand[col_in_m5]

```

```

pred_y1 = mylr1.predict(X1_test)
pred_y2 = mylr2.predict(X2_test)
pred_y3 = mylr3.predict(X3_test)
pred_y4 = mylr4.predict(X4_test)
pred_y5 = mylr5.predict(X5_test)

cm1 = confusion_matrix(y_test,pred_y1,labels=[1,0])
print(cm1)
cm2 = confusion_matrix(y_test,pred_y2,labels=[1,0])
print(cm2)
cm3 = confusion_matrix(y_test,pred_y3,labels=[1,0])
print(cm3)
cm4 = confusion_matrix(y_test,pred_y4,labels=[1,0])
print(cm4)
cm5 = confusion_matrix(y_test,pred_y5,labels=[1,0])
print(cm5)

print("model 1:")
print(accuracy_score(y_test, pred_y1))
print(precision_score(y_test,pred_y1))
print(recall_score(y_test,pred_y1))

print("model 2:")
print(accuracy_score(y_test, pred_y2))
print(precision_score(y_test,pred_y2))
print(recall_score(y_test,pred_y2))

print("model 3:")
print(accuracy_score(y_test, pred_y3))
print(precision_score(y_test,pred_y3))
print(recall_score(y_test,pred_y3))

print("model 4:")
print(accuracy_score(y_test, pred_y4))
print(precision_score(y_test,pred_y4))
print(recall_score(y_test,pred_y4))

print("model 5:")
print(accuracy_score(y_test, pred_y5))
print(precision_score(y_test,pred_y5))
print(recall_score(y_test,pred_y5))

```

6.4.5 Plotting Graph

```

plot_data = {
    "Acc_train": [],
    "Pre_train": [],
    "Rec_train": [],
    "Acc_val": [],
    "Pre_val": [],
    "Rec_val": []
}

```

```

my_columns = ["min temp", "max temp", "rainfall", "evaporation", "sunshine",
              "wind gust dir", "wind gust speed", "wind dir 9am", "wind dir 3pm",
              "wind speed 9am", "wind speed 3pm", "humidity 9am", "humidity 3pm",
              "pressure 9am", "pressure 3pm", "cloud 9am", "cloud 3pm",
              "temp 9am", "temp 3pm", "rain today"]

def get_plot_data(column_to_test, X_train, X_test, y_train, y_test):
    for i in range(len(column_to_test)):
        trial_predictors = itertools.combinations(column_to_test, i + 1)
        for col_comb in trial_predictors:
            X_train_subset = X_train[list(col_comb)]
            X_test_subset = X_test[list(col_comb)]
            y = y_train

            mylr = logreg(C=1e9)
            mylr.fit(X_train_subset, y)

            pred_y = mylr.predict(X_train_subset)
            acc = accuracy_score(y, pred_y)
            prec = precision_score(y, pred_y)
            rec = recall_score(y, pred_y)

            pred_y_test = mylr.predict(X_test_subset)
            acc_val = accuracy_score(y_test, pred_y_test)
            prec_val = precision_score(y_test, pred_y_test)
            rec_val = recall_score(y_test, pred_y_test)

            plot_data["Acc_train"].append(np.mean(acc))
            plot_data["Pre_train"].append(np.mean(prec))
            plot_data["Rec_train"].append(np.mean(rec))
            plot_data["Acc_val"].append(np.mean(acc_val))
            plot_data["Pre_val"].append(np.mean(prec_val))
            plot_data["Rec_val"].append(np.mean(rec_val))

# Assuming you have X_train, X_test, y_train, and y_test defined somewhere before
get_plot_data(my_columns, X_train, X_test, y_train, y_test)

df = pd.DataFrame(plot_data)
print(df)

plt.figure()
plt.plot(df["Acc_train"], "b")
plt.plot(df["Acc_val"], "r")
plt.title("Accuracy")
plt.xlabel("Model Complexity (num of variables)")
plt.ylabel("Mean Accuracy")
plt.legend(["Training data", "Validation data"])
plt.show()

```

4.2 Classification neural network - Tharindu lol if i make it

4.2.1 Methodology

An neural network classifier model was also trained and optimised on matlab. Sequential feature selection was performed to maximise accuracy.

4.2.1.2 Sequential Feature Selection

As for the SVM, forward sequential feature selection was performed by evaluating a model (initially only on one feature) and sequentially adding in a new feature (the one with the next highest correlation to the dependent variable) and calculating the classification loss (1-accuracy of model) until the classification loss stopped decreasing.

Thus, the features 5 6 11 17 X X X X were selected to train the final model.

4.2.1.2 Hyper-parameter optimization

4.2.2 Results

4.2.3 Discussion

7. References:

1. Baku.From:AZERTAC. *Over 100 flights cancelled at Sydney Airport amid heavy rainfall*. Available at: https://azertag.az/en/xeber/over_100_flights_cancelled_at_Sydney_airport_amid_heavy_rainfall-2963301 (Accessed: 22 June 2024)
2. Young, J. (2020) *Rain in Australia*, Kaggle. Available at: <https://www.kaggle.com/datasets/jsphyg/weather-dataset-rattle-package> (Accessed: 12 June 2024).
3. https://en.wikipedia.org/wiki/Decision_tree#:~:text=A%20decision%20tree%20is%20a,only%20contains%20conditional%20control%20statements.
4. <https://medium.com/@dishantkharkar9/about-random-forest-algorithms-62163357db25#:~:text=Random%20Forest%20has%20emerged%20as,popular%20choice%20among%20data%20scientists>.
5. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
6. <https://blog.aci.aero/airports-need-significant-capital-expenditure-to-alleviate-slot-constrain>